

Шпаргалка по тестированию ПО



1. Основные понятия тестирования

Определение тестирования

Тестирование программного обеспечения — процесс исследования ПО с целью получения информации о качестве продукта.

Компоненты тестирования:

- Проверка соответствия требований реализованной функциональности
- Оценка различий между ожидаемым и фактическим состоянием системы
- Анализ и планирование тестовой деятельности
- Разработка тестовых сценариев
- Определение критериев окончания тестирования
- Написание отчетов и рецензирование документации

Цели тестирования

Повысить вероятность корректной работы приложения при любых обстоятельствах

Предотвратить появление дефектов

Предоставить актуальную информацию о состоянии продукта для принятия управленческих решений

Ценность тестирования

- Снижает риски возникновения проблем в продакшене
- Повышает качество системы
- Выполняет контрактные, законодательные и стандартные требования
- Позволяет количественно оценить качество через метрики дефектов
- Обеспечивает уверенность в стабильности продукта

2. Ключевые термины QA

Базовая терминология

- **Ошибка (Error)** — действие человека, приводящее к неправильному результату
- **Дефект (Defect/Bug)** — изъян в компоненте или системе, который может вызвать сбой
- **Отказ (Failure)** — отклонение компонента или системы от ожидаемого выполнения
- **Отчет об ошибке (Bug Report)** — документ, описывающий ситуацию некорректной работы

Роли в команде качества

- **Test Engineer** — специалист, выполняющий проверку соответствия между реальным и ожидаемым поведением
- **QC Quality Control)** — контроль качества продукта, действия над продуктом в процессе разработки
- **QA Quality Assurance)** — обеспечение качества, мероприятия на всех этапах жизненного цикла ПО

3. Принципы тестирования (ISTQB)

Тестирование демонстрирует наличие дефектов — показывает присутствие дефектов, но не доказывает их отсутствие

Исчерпывающее тестирование невозможно — полное тестирование всех комбинаций физически невыполнимо

Раннее тестирование — дефекты, устраненные на ранней стадии, не вызывают последующих проблем

Скопление дефектов — большинство дефектов содержится в небольшом количестве модулей (принцип Парето 80/20)

Парадокс пестицида — повторное выполнение одних тестов не находит новых дефектов

Тестирование зависит от контекста — разные типы ПО требуют различных подходов к тестированию

Заблуждение об отсутствии ошибок — исправление всех дефектов не поможет, если система не решает задачи пользователя

4. Уровни и виды тестирования

Уровни тестирования

Компонентное (Unit Testing) — тестирование отдельных модулей/функций в изоляции

Интеграционное — проверка взаимодействия между компонентами

Системное — тестирование системы в целом как единого продукта

Приемочное (Acceptance) — проверка готовности системы к развертыванию и использованию

Типы тестирования

Функциональное тестирование:

- Smoke тестирование (проверка критичной функциональности)
- Sanity тестирование (быстрая проверка основных функций)
- Регрессионное тестирование (проверка отсутствия новых багов)
- Критического пути (End-to-End)
- Расширенное (Extended)

Нефункциональное тестирование:

- Производительности (Performance)
- Нагрузочное (Load)
- Стресс-тестирование (Stress)
- Стабильности (Stability)
- Удобства использования (Usability)
- Безопасности (Security)
- Совместимости (Compatibility)
- Локализации (Localization)

Подходы к тестированию

- **Черный ящик (Black Box)** — тестирование без знания внутренней структуры
- **Белый ящик (White Box)** — тестирование с полным знанием кода и архитектуры
- **Серый ящик (Grey Box)** — частичное знание внутренней структуры
- **На основе опыта (Experience-based)** — использование знаний и интуиции тестировщика

5. Тестовая документация

Основные артефакты

Тест-план — документ, описывающий объем работ, подход, ресурсы и график тестирования

Чек-лист — список проверок без детальных шагов выполнения

Тест-кейс — набор шагов, условий, входных данных и ожидаемых результатов

Тест-сюит — набор тест-кейсов для тестирования конкретной функциональности

Отчет о тестировании — сводная информация о результатах и метриках **Баг-репорт** — детальное описание найденного дефекта

Структура тест-кейса

- **ID** — уникальный идентификатор (например, TC001)
- **Заголовок** — краткое описание проверки
- **Описание** — детальная информация о цели теста
- **Предусловия** — необходимые условия перед началом
- **Тестовые данные** — входные параметры
- **Шаги к воспроизведению** — последовательность действий
- **Ожидаемый результат** — что должно произойти
- **Фактический результат** — что произошло фактически
- **Приоритет** — важность выполнения теста
- **Статус** — текущее состояние (Pass/Fail/Blocked/Skip)
- **Постусловия** — действия после завершения теста **Вложения** —
 - скриншоты, логи, видео

6. Работа с дефектами

Severity (Критичность)

- **Blocker** — полная блокировка работы приложения, невозможно продолжить тестирование
- **Critical** — нарушение основной бизнес-логики, проблемы безопасности, потеря данных
- **Major** — существенная часть функционала работает некорректно
- **Minor** — незначительные проблемы UI/UX, не влияющие на функционал
- **Trivial** — косметические дефекты, опечатки, мелкие визуальные артефакты

Priority (Приоритет)

- **High** — должно быть исправлено немедленно, блокирует релиз
- **Medium** — требует обязательного решения в текущем спринте
- **Low** — не критично, может быть отложено на следующие релизы

Жизненный цикл дефекта

New — дефект зарегистрирован

Assigned — назначен разработчику

Open — разработчик принял в работу

Fixed — исправлен разработчиком

Retest — передан на повторное тестирование

Verified — подтверждено исправление

Closed — дефект закрыт

Reopened — дефект воспроизводится снова

Rejected — отклонен (не является дефектом) **Duplicate** — дублирует существующий баг

Обязательные атрибуты баг-репорта

ID — уникальный идентификатор

Summary — краткое описание (одна строка)

Description — подробное описание проблемы

Steps to Reproduce — пошаговая инструкция воспроизведения

Actual Result — что произошло фактически

Expected Result — что должно было произойти

Environment — окружение (ОС, браузер, версия приложения, устройство)

Severity — критичность дефекта

Priority — приоритет исправления

Status — текущий статус

Attachments — скриншоты, логи, видео, сетевые запросы

7. Техники тест-дизайна

Техники черного ящика

Эквивалентное разбиение (Equivalence Partitioning):

- Разделение входных данных на классы эквивалентности
- Из каждого класса выбирается один представитель для тестирования
- Позволяет сократить количество тестов без потери покрытия

Анализ граничных значений (Boundary Value Analysis):

- Тестирование на границах допустимых значений
- Проверяются: минимальное, максимальное и граничные значения
-

Комбинируется с эквивалентным разбиением **Таблица решений (Decision Table):**

- Используется для тестирования комбинаций условий
- Содержит условия, действия и правила их сочетаний
- Эффективна для сложной бизнес-логики

Тестирование переходов состояний (State Transition):

- Применяется для систем с различными состояниями
- Проверяются переходы между состояниями
- Включает позитивные и негативные переходы **Pairwise**

Testing (попарное тестирование):

- Комбинаторная техника для сокращения тестов
- Проверяет все возможные пары значений параметров
- Находит 70 80% дефектов при меньшем количестве тестов

Техники белого ящика

- **Statement Coverage** — покрытие операторов (каждая строка кода выполнена)
- **Branch Coverage** — покрытие ветвей (все if/else выполнены)
- **Path Coverage** — покрытие путей (все возможные пути выполнены)

Техники на основе опыта

- **Error Guessing** — предположение об ошибках на основе опыта
- **Exploratory Testing** — исследовательское тестирование без формальной документации
- **Checklist-based Testing** — тестирование на основе чек-листов

8. Жизненный цикл разработки ПО

Модели разработки

Waterfall (Каскадная):

- Последовательное выполнение фаз
- Каждая фаза начинается после завершения предыдущей
-

Тестирование на завершающем этапе **V Model**:

- Параллельная разработка тестов с кодом
- Каждому этапу разработки соответствует этап тестирования Раннее
- обнаружение дефектов **Agile/Scrum**:
- Итеративная разработка короткими спринтами (1-4 недели)
- Постоянное взаимодействие с заказчиком
- Непрерывное тестирование в рамках спринта
- Ретроспективы и адаптация процессов **DevOps**:
- Непрерывная интеграция (CI) и непрерывная доставка (CD)
- Автоматизация тестирования и развертывания
- Быстрая обратная связь
- Культура сотрудничества разработки и операций **Kanban**:
- Визуализация рабочего процесса
- Ограничение незавершенной работы (WIP)
- Непрерывная доставка без жестких итераций

Этапы тестирования в проекте

Анализ требований — изучение спецификаций и требований

Разработка стратегии — определение подхода, ресурсов, рисков

Проектирование тестов — создание тест-кейсов, чек-листов

Подготовка тестового окружения — настройка инфраструктуры

Выполнение тестов — прогон тестов, фиксация результатов

Анализ результатов — оценка покрытия, выявление проблем

Регрессионное тестирование — проверка отсутствия новых дефектов **Отчетность** — предоставление метрик и статуса

9. Управление тестированием

Ключевые метрики

Метрики тест-кейсов:

- **Total Test Cases** — общее количество тест-кейсов
- **Executed** — выполнено
- **Passed** — пройдено успешно
- **Failed** — провалено
- **Blocked** — заблокировано
- **Not Run** — не запущено
- **Pass Rate** — процент успешных тестов: $(Passed / Executed) \times 100\%$ **Метрики**

дефектов:

- **Total Defects** — общее количество дефектов
- **Open Defects** — открытые дефекты
- **Closed Defects** — закрытые дефекты
- **Defect Density** — плотность дефектов на модуль: $(\text{Дефекты} / \text{Размер модуля})$
- **Defect Removal Efficiency** — эффективность удаления: $(\text{Дефекты до релиза} / \text{Все дефекты}) \times 100\%$
- **Defect Leakage** — утечка дефектов в продакшн **Метрики**

покрытия:

- **Requirements Coverage** — покрытие требований тестами
- **Code Coverage** — покрытие кода автотестами
- **Test Coverage** — покрытие функционала тест-кейсами

Критерии входа и выхода

Критерии входа (Entry Criteria):

- Готовность требований и спецификаций
- Доступность тестового окружения
- Наличие тестовых данных
- Готовность сборки для тестирования
- Завершение разработки функционала **Критерии**

выхода (Exit Criteria):

- Выполнены все запланированные тесты
- Достигнут целевой процент покрытия
- Нет открытых критичных и блокирующих дефектов
- Достигнут целевой Pass Rate
- Получено одобрение стейкхолдеров

10. HTTP и API тестирование

Основы HTTP

HTTP методы:

- **GET** — получение данных с сервера (идемпотентный, безопасный)
- **POST** — отправка данных для создания ресурса (не идемпотентный)
- **PUT** — полное обновление существующего ресурса (идемпотентный)
- **PATCH** — частичное обновление ресурса (идемпотентный)
- **DELETE** — удаление ресурса (идемпотентный)
- **HEAD** — получение заголовков без тела ответа
- **OPTIONS** — получение доступных методов для ресурса

Коды состояния HTTP

Код	Значение	Описание
200	OK	Успешный запрос
201	Created	Ресурс создан
204	No Content	Успешно, без тела ответа
301	Moved Permanently	Постоянное перенаправление
302	Found	Временное перенаправление
400	Bad Request	Ошибка в запросе клиента
401	Unauthorized	Требуется аутентификация
403	Forbidden	Доступ запрещен
404	Not Found	Ресурс не найден
405	Method Not Allowed	Метод не разрешен
409	Conflict	Конфликт при обновлении

422	Unprocessable Entity	Невалидные данные
429	Too Many Requests	Превышен лимит запросов
500	Internal Server Error	Внутренняя ошибка сервера
502	Bad Gateway	Ошибка шлюза
Код	Значение	Описание
503	Service Unavailable	Сервис недоступен
504	Gateway Timeout	Таймаут шлюза

REST API тестирование

Что проверять:

- Корректность HTTP статус-кодов
- Структуру JSON/XML ответа (schema validation)
- Валидацию входных данных
- Обработку ошибок и граничных значений
- Авторизацию и аутентификацию (токены, API keys)
- Производительность и время ответа
- Идемпотентность методов
- Rate limiting
- CORS политики
- Версионирование API

Инструменты:

- Postman
- REST Assured
- SoapUI
- JMeter
- Insomnia
- curl

SOAP vs REST

Параметр	REST	SOAP
Протокол	HTTP/HTTPS	HTTP, SMTP, TCP

Формат	JSON, XML	Только XML
Сложность	Проще	Сложнее
Скорость	Быстрее	Медленнее
Безопасность	HTTPS, OAuth	WS Security

11. Тестирование мобильных приложений

Типы мобильных приложений

- **Native** — разработано для конкретной ОС (Swift/Objective-C для iOS, Kotlin/Java для Android)
- **Hybrid** — веб-приложение в нативной оболочке (Ionic, Cordova)
- **Cross-platform** — единая кодовая база для разных ОС (React Native, Flutter)

Специфика мобильного тестирования

Проверки совместимости:

- Различные версии ОС (iOS 14 17, Android 10 14)
- Разные разрешения экранов и размеры устройств
- Различные производители устройств **Функциональные**

проверки:

- Установка и удаление приложения
- Обновление приложения
- Прерывания (звонки, SMS, уведомления)
- Сворачивание/разворачивание приложения
- Переход между приложениями
- Работа в фоновом режиме
- Ротация экрана

Проверки производительности:

- Работа при плохом интернет-соединении 2G, 3G, 4G, 5G)
- Переключение между Wi-Fi и мобильными данными
- Работа в оффлайн режиме
- Потребление батареи
- Использование памяти и CPU
- Время запуска приложения **Специфические**

проверки:

- Работа с push-уведомлениями
- Интеграция с камерой, GPS, сенсорами
- Жесты (свайп, пинч, долгое нажатие)
- Поддержка разных языков
- Accessibility (доступность)

12. Автоматизация тестирования

Когда автоматизировать

Подходит для автоматизации:

- Регрессионные тесты
- Smoke тесты
- Повторяющиеся тест-кейсы
- Проверки на больших объемах данных
- Нагрузочное тестирование
- Стабильный функционал

Не подходит для автоматизации:

- Одноразовые тесты
- Нестабильный функционал
- Тесты UX/UI дизайна
- Исследовательское тестирование
- Тесты с высокой стоимостью поддержки

Популярные фреймворки Web:

- Selenium WebDriver
- Playwright Cypress
- Puppeteer
- TestCafe
- **Mobile:**
- Appium
- Espresso Android)
- XCUITest (iOS)
- Detox

API

- REST Assured
- Postman/Newman Karate
- Framework
- Pytest + Requests

Unit:

- JUnit Java
- PyTest Python
- Jest JavaScript) NUnit C#
-

Pyramid тестирования

Unit тесты 70% — быстрые, изолированные, низкая стоимость

Integration тесты 20% — проверка взаимодействия компонентов

E2E тесты 10% — медленные, проверка полных сценариев, высокая стоимость

Преимущества и риски

Преимущества:

- Экономия времени на повторяющихся проверках
- Повышение точности тестирования
- Быстрая обратная связь в CI/CD
- Возможность запуска в ночное время
- Параллельное выполнение тестов **Риски и**

ограничения:

- Высокие первоначальные затраты
- Необходимость постоянной поддержки тестов
- Зависимость от инструментов
- Ложно-положительные и ложно-отрицательные результаты Не
- заменяет мануальное тестирование полностью

13. Тестирование безопасности

OWASP Top 10 (основные уязвимости)

Broken Access Control — нарушение контроля доступа

Cryptographic Failures — проблемы с шифрованием данных

Injection — SQL, NoSQL, OS, LDAP инъекции

Insecure Design — небезопасный дизайн архитектуры

Security Misconfiguration — неправильная настройка безопасности

Vulnerable Components — использование уязвимых компонентов

Identification and Authentication Failures — проблемы аутентификации

Software and Data Integrity Failures — нарушение целостности

Security Logging and Monitoring Failures — недостаточное логирование

Server-Side Request Forgery SSRF — подделка запросов на стороне сервера

Распространенные атаки

SQL инъекции:

- Внедрение SQL кода через пользовательский ввод
- Проверка: использование спецсимволов ' OR '1'='1
-

Защита: параметризованные запросы, валидация ввода **XSS Cross-Site**

Scripting):

- Внедрение JavaScript-кода на страницу
- Типы: Reflected, Stored, DOM-based

- Проверка : ввод тегов `<script>alert('XSS')</script>`

-

Защита: санитизация вывода, Content Security Policy **CSRF Cross-Site Request Forgery):**

- Выполнение нежелательных действий от имени пользователя
- Проверка: отправка запросов без токена
- Защита: CSRF токены, проверка Referer **Broken**

Authentication:

- Слабые пароли, хранение паролей в открытом виде
- Отсутствие блокировки после неудачных попыток Незащищенные токены
- сессий

Методы тестирования безопасности

Аутентификация и авторизация:

- Проверка политик паролей
- Тестирование ролевого доступа (RBAC)
- Проверка сессий и токенов
- Тестирование выхода из системы **Тестирование**

ввода:

- Валидация длины полей
- Проверка на SQL, XSS, Command инъекции
- Загрузка вредоносных файлов
- Unicode и специальные символы **Шифрование:**

- Использование HTTPS
- Проверка сертификатов SSL/TLS
- Безопасное хранение паролей (хеширование) Шифрование
- чувствительных данных

14. SQL для тестировщиков

Базовые команды

SELECT (выборка данных):

```
SELECT * FROM users WHERE id = 1;  
SELECT name, email FROM users;  
SELECT COUNT (*) FROM orders;
```

INSERT (добавление):

```
INSERT INTO users (name, email) VALUES ('John', 'john@test.com');
```

UPDATE (обновление):

```
UPDATE users SET email = 'new@test.com' WHERE id = 1;
```

DELETE (удаление):

```
DELETE FROM users WHERE id = 1;
```

Операторы и функции WHERE

(фильтрация):

- =, !=, <>, >, <, >=, <=
- AND, OR, NOT
- IN, NOT IN
- LIKE (поиск по шаблону: % — любые символы, _ — один символ)
- BETWEEN
- IS NULL, IS NOT NULL

JOIN (объединение таблиц):

- `INNER JOIN` — записи, присутствующие в обеих таблицах
- `LEFT JOIN` — все записи из левой таблицы
- `RIGHT JOIN` — все записи из правой таблицы
- `FULL JOIN` — все записи из обеих таблиц **Агрегатные**

функции:

- `COUNT()` — количество записей
- `SUM()` — сумма
- `AVG()` — среднее
- `MAX()` — максимум
- `MIN()` — минимум

GROUP BY и ORDER BY

```
SELECT status, COUNT(*) FROM orders GROUP BY status;  
SELECT * FROM users ORDER BY created_at DESC;
```

15. Git для тестировщиков**Основные команды Базовые****операции:**

```
git clone <url> # клонирование репозитория  
git status # статус изменений  
git add . # добавить все изменения  
git add <file> # добавить файл  
git commit -m "message" # зафиксировать изменения  
git push # отправить на сервер  
git pull # получить изменения
```

Работа с ветками:

```
git branch # список веток  
git branch <name> # создать ветку  
git checkout <branch> # переключиться на ветку  
git checkout -b <branch> # создать и переключиться на ветку  
git merge <branch> # слить ветку  
git branch -d <branch> # удалить ветку
```

Просмотр истории:

```
git log                                # история
коммитов git log --oneline             #
краткая история git diff
# изменения git show <commit>         #
детали коммита
```

Откат изменений:

```
git reset --hard HEAD                 # откатить все
изменения git revert <commit>         # создать
коммит отмены git checkout -- <file> #
отменить изменения файла
```

16. Инструменты тестировщика**Управление тестами**

- Jira
- TestRail
- Zephyr
- Qase
- TestIT
- PractiTest

Баг-трекинг

- Jira
- Bugzilla
- Mantis
- Redmine
- YouTrack

API тестирование

- Postman
- Insomnia
- SoapUI
- Swagger
- Charles Proxy Fiddler
-

Тестирование производительности

- JMeter
- Gatling
- LoadRunner
- K6
- Artillery

Мониторинг и логи

- Kibana
- Grafana
- Splunk
- New Relic Datadog
-

Инструменты для мобильного тестирования

- Android Studio (эмуляторы)
- Xcode (симуляторы)
- BrowserStack
- Sauce Labs
- Firebase Test Lab

Работа с базами данных

- DBeaver MySQL
- Workbench
- pgAdmin
- SQL Server Management Studio

Скриншоты и видео

- Jing
- ShareX
- OBS Studio
- Snagit
- LICEcap

17. Полезные сокращения и термины

Общие термины

- **SDLC** — Software Development Life Cycle (жизненный цикл разработки ПО)
- **STLC** — Software Testing Life Cycle (жизненный цикл тестирования)
- **CI/CD** — Continuous Integration/Continuous Delivery
- **SUT** — System Under Test (тестируемая система)
- **AUT** — Application Under Test (тестируемое приложение)
- **DoD** — Definition of Done (критерии готовности)
- **UAT** — User Acceptance Testing (приемочное тестирование)
- **E2E** — End-to-End (сквозное тестирование)
- **MVP** — Minimum Viable Product (минимально жизнеспособный продукт) **PoC** — Proof of Concept (доказательство концепции)

Аббревиатуры методологий

- **TDD** — Test-Driven Development (разработка через тестирование)
- **BDD** — Behavior-Driven Development (разработка через поведение)
- **ATDD** — Acceptance Test-Driven Development **DRY** — Don't Repeat Yourself (не повторяйся)
- **KISS** — Keep It Simple, Stupid (делай проще)
- **YAGNI** — You Aren't Gonna Need It (тебе это не понадобится)

Метрики и показатели

- **RCA** — Root Cause Analysis (анализ первопричины)
- **SLA** — Service Level Agreement (соглашение об уровне сервиса)
- **TTM** — Time To Market (время выхода на рынок)
- **MTBF** — Mean Time Between Failures (среднее время между отказами) **MTTR** — Mean Time To Repair (среднее время восстановления)

18. Чек-лист веб-тестирования

Функциональное тестирование

- Все ссылки ведут на корректные страницы
- Формы отправляются корректно
- Валидация полей работает правильно
- Обязательные поля помечены
- Сообщения об ошибках информативны
- Кнопки "Назад" и "Вперед" работают
- Cookies сохраняются корректно
- Тайм-аут сессии работает
- Поиск возвращает релевантные результаты
- Пагинация работает корректно

UI/UX тестирование

- Дизайн консистентен на всех страницах
- Шрифты читаемы
- Цветовая схема приятна для глаз
- Изображения загружаются корректно
- Альтернативный текст присутствует
- Выравнивание элементов правильное
- Нет лишних скроллбаров
- Курсор изменяется на интерактивных элементах Анимации
- работают плавно

Совместимость

- Работа в Chrome, Firefox, Safari, Edge
- Корректное отображение на мобильных устройствах
- Адаптивный дизайн работает
- Проверка на разных разрешениях экрана
- Работа на разных ОС

Производительность

- Страницы загружаются быстро (< 3 сек)
- Нет медленных запросов
- Изображения оптимизированы
- CSS и JS минифицированы
- Кеширование работает
- CDN используется для статики

Безопасность

- HTTPS используется
- Пароли не отображаются
- SQL инъекции не работают
- XSS атаки блокируются
- Доступ к admin-панели защищен
- Сессия завершается при выходе
- Чувствительные данные не в U